



# Sistema de Assinatura Digital com RSA-OAEP e RSA-PSS

Trabalho de Implementação 2 — CIC0201 Segurança Computacional — Universidade de Brasília

Ana Luísa Reis Nascente  
Matrícula: 211045688

Gabriel de Sousa  
Matrícula: 211056000

Marina Pimentel Moreno  
Matrícula: 222014071

Pedro de Paula Campos  
Matrícula: 231036050

**Resumo**—Implementamos RSA-OAEP e RSA-PSS com SHA3-256/MGF1 e chaves de 2048 bits via Miller-Rabin, sem bibliotecas criptográficas de alto nível. Testes automatizados validaram o limite de 190 bytes do OAEP, a natureza probabilística do PSS e a matriz de adulteração da Parte IV; OAEP e PSS interoperam, nos dois sentidos, com bibliotecas de referência.

**Index Terms**—RSA-OAEP, RSA-PSS, Assinatura Digital, SHA3-256

## I. INTRODUÇÃO

A criptografia assimétrica é o pilar para estabelecer canais seguros, autenticidade e não repúdio em sistemas computacionais modernos. No âmbito da disciplina CIC0201 — Segurança Computacional, o Trabalho de Implementação 2 propõe a construção de um ambiente integrado para geração de chaves, cifragem de mensagens curtas e assinatura/verificação segura de arquivos utilizando o criptossistema RSA [1].

Diferentemente do uso ingênuo do RSA (“textbook RSA”), cujas propriedades matemáticas determinísticas e multiplicativas o tornam vulnerável a ataques de texto claro escolhido e a forjamento de assinaturas, aplicações práticas exigem esquemas de codificação probabilísticos padronizados pela norma PKCS #1 v2.2 (RFC 8017) [2]. Empregou-se o RSA-OAEP (*Optimal Asymmetric Encryption Padding*) para confidencialidade com segurança IND-CCA2 e o RSA-PSS (*Probabilistic Signature Scheme*) para integridade e autenticidade — explicitamente sem tratar a assinatura como uma simples “cifragem do hash”.

O sistema foi estruturado de forma modular em Python 3, restrito à biblioteca padrão para os cálculos numéricos e transformações de bits (`secrets` para aleatoriedade criptográfica e `hashlib.sha3_256` para o digest) [3]; bibliotecas consolidadas foram usadas apenas no teste adicional de interoperabilidade, conforme permitido pelo enunciado.

Este relatório está organizado da seguinte forma: a Seção II descreve a fundamentação teórica, o fluxo dos algoritmos e a análise de segurança (Parte V); a Seção III detalha o ambiente experimental e a análise de resultados de cada parte, incluindo a matriz de adulteração e os ensaios de interoperabilidade; e a Seção IV sintetiza as conclusões técnicas alcançadas.

## II. FUNDAMENTAÇÃO TEÓRICA E ARQUITETURA

### A. RSA e Geração de Chaves com Miller-Rabin

O algoritmo RSA fundamenta-se na dificuldade computacional de fatorar inteiros compostos grandes obtidos pelo produto de dois primos aleatórios  $p$  e  $q$  [5]. A geração do par de chaves segue os passos: (1) gerar  $p$  e  $q$  com metade dos bits do módulo, distintos e afastados em pelo menos  $2^{bits/2-100}$  (FIPS 186-5, Apêndice A.1.1); (2) calcular  $n = pq$  e  $\lambda(n) = \text{mmc}(p-1, q-1)$ ; (3) fixar  $e = 65537$  e calcular  $d = e^{-1} \bmod \lambda(n)$  pelo algoritmo estendido de Euclides; (4) derivar os parâmetros CRT  $d_p = d \bmod (p-1)$ ,  $d_q = d \bmod (q-1)$  e  $q_{inv} = q^{-1} \bmod p$ , usados para acelerar as operações com a chave privada.

A primalidade de  $p$  e  $q$  é verificada pelo teste probabilístico de Miller-Rabin [4]. Dado um candidato ímpar  $n$ , escreve-se  $n-1 = 2^s d$ , com  $d$  ímpar. Para uma base aleatória  $a \in [2, n-2]$ , calcula-se  $x = a^d \bmod n$ ; se  $x = 1$  ou  $x = n-1$ ,  $n$  passa na rodada, senão repete-se o quadrado  $x \leftarrow x^2 \bmod n$  por  $s-1$  vezes. Antes do teste probabilístico, um crivo de Eratóstenes com limite  $10^3$  descarta rapidamente a maioria dos candidatos compostos sem exponenciação modular. O número de rodadas não é fixo: a implementação usa a Tabela B.1 da FIPS 186-5, que, para primos de 1024 bits (módulo de 2048 bits) e probabilidade de erro  $\leq 2^{-100}$ , exige apenas **4 rodadas** — muito menos que o limite genérico de pior caso ( $4^{-t}$  por rodada, válido para qualquer composto), pois a fração de candidatos aleatórios que sobrevivem a  $t$  rodadas cai mais rápido que essa cota [7].

### B. Função Hash SHA3-256 e MGF1

O padrão SHA-3 adota a construção de esponja sobre a permutação Keccak-f[1600] [3], produzindo resumos de 256 bits (32 bytes) com alta resistência a colisão e pré-imagem. A Função de Geração de Máscara MGF1 (RFC 8017 [2]) expande uma semente  $Z$  para uma máscara de tamanho arbitrário  $L$  encadeando hashes indexados por um contador de 4 bytes:

$$T = \text{SHA3-256}(Z\|c_0) \|\text{SHA3-256}(Z\|c_1) \|\dots \quad (1)$$

truncado para os primeiros  $L$  bytes.

### C. RSA-OAEP

O esquema EME-OAEP impede ataques de texto claro escolhido com uma rede de Feistel de duas rodadas aplicada à mensagem antes da exponenciação modular [2]. Para módulo  $n$  de  $k = 256$  bytes (2048 bits) e  $hLen = 32$  bytes: a mensagem  $M$  deve ter  $mLen \leq k - 2hLen - 2 = 190$  bytes; concatena-se  $lHash = \text{SHA3-256}(L)$  (rótulo, vazio por padrão), o preenchimento  $PS$ , o separador  $0x01$  e  $M$ , formando o bloco  $DB$  de  $k - hLen - 1 = 223$  bytes; gera-se uma semente aleatória  $seed$  de 32 bytes (`secrets.token_bytes`); calcula-se  $maskedDB = DB \oplus \text{MGF1}(seed, 223)$  e  $maskedSeed = seed \oplus \text{MGF1}(maskedDB, 32)$ ; o bloco codificado  $EM = 0x00 || maskedSeed || maskedDB$  é cifrado como  $c = m^e \bmod n$ . Na decifragem, qualquer desvio no tamanho, no byte inicial, no  $lHash$  ou no separador resulta na mesma rejeição genérica; a decodificação percorre todo o bloco sem sair antecipadamente e testa as condições de uma só vez, para não criar um oráculo de padding [10].

### D. RSA-PSS

O esquema EMSA-PSS associa um *salt* aleatório ao digest do arquivo para tornar a assinatura probabilística [4]. Com  $sLen = 32$  bytes: calcula-se  $mHash = \text{SHA3-256}(M)$ ; forma-se  $M' = 0x00_s || mHash || salt$  e  $H = \text{SHA3-256}(M')$ ; monta-se  $DB = PS || 0x01 || salt$ ; calcula-se  $maskedDB = DB \oplus \text{MGF1}(H, k - hLen - 1)$ , zerando o bit mais significativo; o bloco final é  $EM = maskedDB || H || 0xBC$  e a assinatura é  $s = EM^d \bmod n$  (primitiva RSASP1, com CRT). Na verificação, extrai-se  $H$  e  $maskedDB$  de  $EM = s^e \bmod n$ , remove-se a máscara, recupera-se o *salt*, reconstrói-se  $M'$  e compara-se  $\text{SHA3-256}(M')$  com  $H$ .

### E. Análise de Segurança (Parte V)

1) *Fragilidades do RSA puro (textbook RSA)*: Aplicar RSA diretamente sobre  $m$  ( $c = m^e \bmod n$  ou  $s = m^d \bmod n$ ) é inseguro: (i) **determinismo** – mensagens idênticas geram ciphertexts/assinaturas idênticos, permitindo ataques de dicionário; (ii) **maleabilidade multiplicativa** – como  $(m_1 m_2)^e \equiv m_1^e m_2^e \pmod{n}$ , um atacante combina ciphertexts ou assinaturas válidos para forjar  $m_1 m_2$ ; (iii) **expoente pequeno** – com  $e = 3$  e mensagem curta ( $m^e < n$ ), basta extrair a raiz cúbica inteira do ciphertext (ataque de Håstad).

2) *Papel do OAEP e do PSS*: O OAEP elimina o determinismo e a maleabilidade injetando aleatoriedade e espalhando a mensagem por todo o bloco codificado, com segurança IND-CCA2 provada no modelo de oráculo aleatório [8]. O PSS estende o hash com *salt* aleatório e tem redução de segurança *tight* ao problema RSA no mesmo modelo [9] – ao contrário do PKCS#1 v1.5, historicamente vulnerável a ataques como o de Bleichenbacher (2006) [2].

3) *RSA-PSS versus Ed25519*: A Tabela I contrasta o RSA-PSS implementado com a alternativa moderna em curvas elípticas Ed25519 (RFC 8032) [6]. Ambos são quebrados por um computador quântico suficiente grande via algoritmo de Shor; alternativas pós-quânticas incluem ML-DSA (FIPS 204) e SLH-DSA (FIPS 205).

Tabela I  
RSA-PSS (2048 BITS) versus Ed25519

| Característica      | RSA-PSS                    | Ed25519              |
|---------------------|----------------------------|----------------------|
| Problema difícil    | Fatoração / RSA            | Log discreto (curva) |
| Segurança           | ~112 bits                  | ~128 bits            |
| Chave pública       | 256 bytes                  | 32 bytes             |
| Assinatura          | 256 bytes                  | 64 bytes             |
| Aleatoriedade       | <i>Salt</i> por assinatura | Nonce determinístico |
| Assinar / verificar | Lento / rápido             | Rápido / rápido      |

## III. AMBIENTE EXPERIMENTAL E ANÁLISE DE RESULTADOS

### A. Descrição do Cenário

O projeto foi organizado em módulos independentes, um por parte do enunciado: `parte1_chaves/` (aritmética modular, Miller-Rabin, geração e formato PEM-like das chaves); `parte2_oaep/` (MGF1 e codificação/decodificação OAEP); `parte3_pss/` (EMSA-PSS e assinatura de arquivos em Base64); `parte4_verificacao/` (parsing estrito da estrutura JSON `rsa-segcomp-assinatura-v1` e verificação de integridade); e `rsa_cli.py`, uma CLI integrada que expõe os subcomandos `gerar-chaves`, `cifrar`, `decifrar`, `assinar` e `verificar`. Cada módulo tem suíte própria de testes automatizados com `unittest`, além de testes de interoperabilidade com as bibliotecas `cryptography` (PSS) e `pycryptodome` (OAEP), permitidas pelo enunciado apenas para essa verificação cruzada.

Os experimentos a seguir foram executados em uma máquina com processador AMD Ryzen 9 5900XT (16 núcleos/32 threads), 32 GB de RAM, sistema operacional Ubuntu 24.04.4 LTS e Python 3.12.3, sem qualquer aceleração de hardware específica para aritmética de precisão arbitrária (apenas o tipo `int` nativo do CPython).

### B. Geração de Chaves (Parte I)

A Figura 5 (linha “1.”) mostra a geração de um par de chaves RSA-2048 pela CLI. Em 8 execuções de `gerar_par_chaves()`, o tempo total (geração de  $p$ ,  $q$  e dos parâmetros CRT) foi, em média, **0,365 s** (mínimo 0,046 s, máximo 0,823 s – Figura 1), variação esperada porque o número de candidatos ímpares testados até achar um primo de 1024 bits é uma variável aleatória: nas mesmas execuções, cada primo exigiu entre 14 e 794 candidatos (Figura 2), com apenas **4 rodadas** de Miller-Rabin por candidato, conforme a Tabela B.1 da FIPS 186-5 (Seção II). Nenhuma execução produziu falso positivo: todas as chaves passaram nos testes de importação/exportação (`test_exportacao_importacao`) e nas primitivas RSA de ida e volta.

### C. Cifragem e Decifragem OAEP (Parte II)

Testes de ida e volta com mensagens de 0 a 190 bytes confirmaram integridade total. O limite foi verificado experimentalmente: cifrar 191 bytes retorna

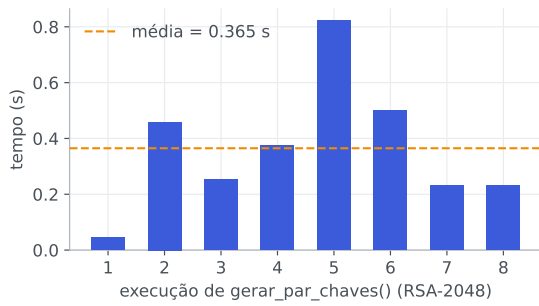


Figura 1. Tempo de geração de chaves RSA-2048 em 8 execuções.

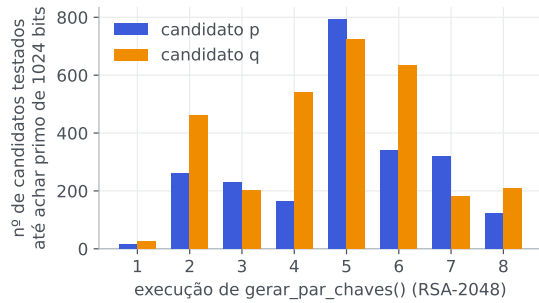


Figura 2. Candidatos ímpares testados até obter cada primo de 1024 bits.

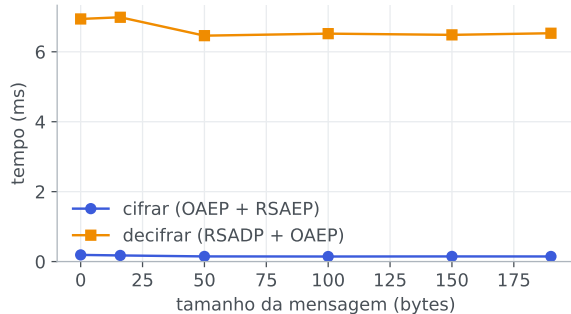


Figura 3. Tempo de cifrar/decifrar RSA-OAEP por tamanho de mensagem.

ErroParametro: mensagem longa demais: máximo de 190 bytes, exatamente o previsto por  $k - 2hLen - 2$ . A Figura 3 mostra que o tempo de cifrar é praticamente constante ( $\approx 0,15$  ms, dominado pela exponenciação com  $e = 65537$ ) e independe do tamanho da mensagem, enquanto decifrar é  $\approx 40\times$  mais lento ( $\approx 6,5$  ms), pois usa o expoente privado completo com CRT – assimetria clássica do RSA. Invertendo um único bit do ciphertext antes de decifrar, a implementação rejeitou com `ErroDecifracao: falha na decifração em todos os casos` (Figura 5, linha “2.”), sem revelar qual verificação do padding falhou.

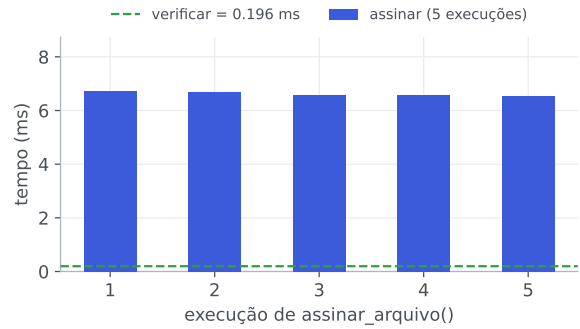


Figura 4. Tempo de assinar (5 execuções) versus verificar (RSA-PSS).

Tabela II  
MATRIZ DE TESTES DE ADULTERAÇÃO (PARTE IV)

| Cenário                            | Resultado | Mecanismo   |
|------------------------------------|-----------|-------------|
| Original (controle)                | Válido    | PSS-VERIFY  |
| (a) 1 byte do arquivo alterado     | Inválido  | PSS-VERIFY  |
| (b) 1 byte da assinatura alterado  | Inválido  | PSS-VERIFY  |
| (c) chave pública diferente        | Inválido  | PSS-VERIFY  |
| (d) delimitador do .sig corrompido | Inválido  | ErroFormato |

#### D. Assinatura Probabilística RSA-PSS (Parte III)

Assinando o mesmo arquivo cinco vezes com a mesma chave privada, as cinco strings Base64 resultantes foram todas distintas entre si (nenhuma repetição), confirmando a natureza probabilística do *salt* de 32 bytes – e não uma simples “cifragem do hash”, que seria determinística. O tempo de assinar (média 6,6 ms) e de verificar (0,196 ms,  $\approx 34\times$  mais rápido) reproduzem a mesma assimetria observada no OAEP (Figura 4), pois assinar usa a chave privada e verificar usa apenas o expoente público.

#### E. Verificação e Matriz de Adulteração (Parte IV)

A Tabela II resume a suíte `test_adulteracao.py` e a reprodução manual via CLI, mostrada na Figura 5 (linha “4.”). Em todos os cenários de adulteração a verificação retornou ASSINATURA INVÁLIDA; apenas o par arquivo/assinatura original e sem alterações foi aceito.

Em todos os casos, `verificar_arquivo` captura as exceções internamente e retorna `ResultadoVerificacao(valido=False, ...)`, de modo que a CLI nunca é interrompida abruptamente por entrada maliciosa ou malformada – ela retorna o código de saída 2 e uma mensagem clara (Figura 5).

#### F. Interoperabilidade e Suíte Automatizada

Os testes adicionais de interoperabilidade usam bibliotecas consolidadas apenas para conferir a saída da implementação, conforme permitido pelo enunciado, sempre nos dois sentidos. Para o PSS, com a `cryptography`: (1) assinaturas

```
terminal - bash exemplos/demo.sh

$ bash exemplos/demo.sh
== 1. Geração de chaves RSA-2048 (Miller-Rabin) ==
Par de chaves RSA-2048 gravado em exemplos/saida/chaves/

== 2. RSA-OAEP ==
Ciphertext (Base64): cfYtY2GD0PV03CQrDxFtVkfXh4GqnWTWR3KI4eGu3cbgpBN7ilik0e/NIf2M...
senha: correct horse battery staple
Ciphertext adulterado:
Erro: falha na decifração

== 3. Assinatura RSA-PSS ==
Assinatura gravada em exemplos/saida/documento.txt.sig
-----BEGIN RSA-SECCOMP SIGNATURE-----
{"formato": "rsa-seccomp-assinatura-v1", "algoritmo": "RSASSA-PSS", "hash": "SHA3-256", "mgf": "MGF1-SHA3-256",
"tamanho_salt": 32, "arquivo": "documento.txt", "assinatura": "RN/BvcLmdiYtMzk0b0eFzIvnNhfD5KZQczDm/vG+7ffUJB8i1
iIuKhDQLF7s2LhNqFowtExcBjJkFfMCAAwR7yJtf+Yf0XL+aRbukyBP0by3noF8J1GaRZGLn02vn5qFyvxye0l3H0RY+eGRf1nwbLxcL2Tr33zGW
glpxF7/bokT7RtUGIh0jdeivDzp9ubj2dW7L1PqC3hShcaFxDsIxkYp9D9EHbjWsajQtu1W15NZm6/h9CGR0yuiImzwwkNLRvRVd5niosWCltidx
+XfEJYt84UnKmoF0M6YTMvy0L9RyMDzm6S1K//udEj+1CKAif4mS1E0LS0bzQu4Z0l8pw=="}
-----END RSA-SECCOMP SIGNATURE-----

== 4. Verificação ==
-- original:
ASSINATURA VÁLIDA: o arquivo é íntegro e foi assinado pela chave informada.
-- (a) um byte do arquivo alterado (1.000 -> 9.000):
ASSINATURA INVÁLIDA: o arquivo foi alterado, a assinatura foi adulterada ou a chave não corresponde.
-- (b) um byte da assinatura alterado:
ASSINATURA INVÁLIDA: o arquivo foi alterado, a assinatura foi adulterada ou a chave não corresponde.
-- (c) outra chave pública:
ASSINATURA INVÁLIDA: o arquivo foi alterado, a assinatura foi adulterada ou a chave não corresponde.
```

Figura 5. `bash exemplos/demo.sh`: geração de chaves, cifra/decifra OAEP com ciphertext adulterado, assinatura PSS e os três testes de adulteração da Parte IV, em execução real.

geradas pela nossa implementação são aceitas pela biblioteca; e (2) assinaturas produzidas pela biblioteca são aceitas por `verificar_digest` e, sobre um arquivo real, por `verificar_arquivo`. Para o OAEP, a `cryptography` não aceita SHA3-256 como hash do OAEP, por isso a conferência usa a `pycryptodome`: ciphertexts gerados por nós (de 0 a 190 bytes) são decifrados pela biblioteca e vice-versa, inclusive com rótulo *L* não vazio. A concordância dos digests SHA3-256, das máscaras MGF1 e do *trailer 0xBC* confirma a conformidade com a RFC 8017.

A suíte completa, reproduzida na Figura 7, executou **52 testes, todos aprovados**, dos quais 6 de interoperabilidade. A Figura 6 discrimina os testes por parte do trabalho. Sem as bibliotecas opcionais instaladas, os testes de interoperabilidade são pulados automaticamente e os demais continuam executando.

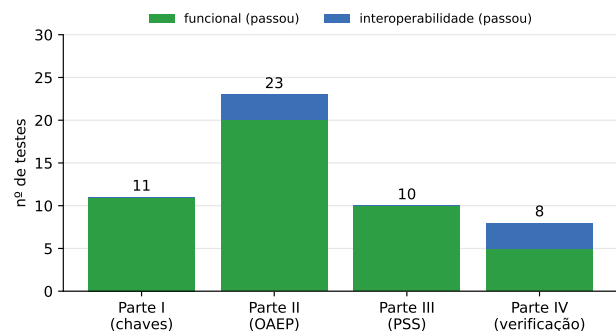


Figura 6. Testes aprovados por parte (52 no total; em azul, os de interoperabilidade).

```
terminal - saída automatizada de testes (unittest)

$ python3 -m unittest discover -p "test_*.py" -v
test_exp_modular (partes.chaves.test_partes.TestArithmetic) ... ok
test_inverso_modular (partes.chaves.test_partes.TestArithmetic) ... ok
test_md5_estendido (partes.chaves.test_partes.TestArithmetic) ... ok
test_exportacao_importacao (partes.chaves.test_partes.TestChaves) ... ok
test_exportacao_nuliformada (partes.chaves.test_partes.TestChaves) ... ok
test_parametros (partes.chaves.test_partes.TestChaves) ... ok
test_primeiras_ida_e_volta (partes.chaves.test_partes.TestChaves) ... ok
test_rejeita_mensagem (partes.chaves.test_partes.TestChaves) ... ok
test_como_milhares (partes.chaves.test_partes.TestMillerRabin) ... ok
test_primeiro_garoto_tem_tamanho_exato (partes.chaves.test_partes.TestMillerRabin) ... ok
test_primeiros_combinados (partes.chaves.test_partes.TestMillerRabin) ... ok
[... 36 testes intermediários omitidos por espaço - ver figura 6 ...]
test_estrutura_nuliformada (partes.verificacao.testes.test_adulteracao.TestAdulteracao) ... ok
test_original_valido (partes.verificacao.testes.test_adulteracao.TestAdulteracao) ... ok
test_arquivoassinado_pela_biblioteca_possuio_verificar_arquivo (partes.verificacao.testes.test_interoperabilidade.TestInteroperabilidade) ... ok
test_pss_da_biblioteca_verificado_por_mim (partes.verificacao.testes.test_interoperabilidade.TestInteroperabilidade) ... ok
test_pss_verificado_pela_biblioteca (partes.verificacao.testes.test_interoperabilidade.TestInteroperabilidade) ... ok
.....
Ran 52 tests in 4.772s
OK
```

Figura 7. Execução real de `python3 -m unittest discover -p "test_*.py" -v`.

#### IV. CONCLUSÕES

O sistema cumpriu as Partes I a V sem bibliotecas criptográficas para geração de chaves, operações RSA, OAEP, MGF1 ou PSS. As chaves RSA-2048 foram geradas em 0,365 s em média, com 4 rodadas de Miller-Rabin por candidato. O OAEP rejeitou mensagens acima de 190 bytes e qualquer ciphertext adulterado com um erro único, e o PSS gerou assinaturas distintas para o mesmo arquivo. Todos os cenários de adulteração foram rejeitados, e os 52 testes passaram, com interoperabilidade nos dois sentidos. A segurança vem do padding probabilístico, não do RSA isolado.

#### REFERÊNCIAS

- [1] P. S. Barreto, “Trabalho de Implementação 2 – Sistema de Assinatura Digital e Verificação Segura de Arquivos,” CIC0201 Segurança Computacional, Universidade de Brasília, 2026.
- [2] K. Moriarty, B. Kaliski, J. Jonsson e A. Rusch, “PKCS #1: RSA Cryptography Specifications Version 2.2,” RFC 8017, IETF, 2016.
- [3] NIST, “SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions,” FIPS PUB 202, 2015.
- [4] NIST, “Digital Signature Standard (DSS),” FIPS PUB 186-5, 2023.
- [5] NIST, “Recommendation for Pair-Wise Key-Establishment Using Integer Factorization Cryptography,” SP 800-56B Rev. 2, 2019.
- [6] S. Josefsson e I. Liusvaara, “Edwards-Curve Digital Signature Algorithm (EdDSA),” RFC 8032, IETF, 2017.
- [7] I. Damgård, P. Landrock e C. Pomerance, “Average case error estimates for the strong probable prime test,” *Mathematics of Computation*, vol. 61, n. 203, pp. 177–194, 1993.
- [8] M. Bellare e P. Rogaway, “Optimal Asymmetric Encryption,” em *Advances in Cryptology – EUROCRYPT ’94*, LNCS 950, Springer, 1995, pp. 92–111.
- [9] M. Bellare e P. Rogaway, “The Exact Security of Digital Signatures – How to Sign with RSA and Rabin,” em *Advances in Cryptology – EUROCRYPT ’96*, LNCS 1070, Springer, 1996, pp. 399–416.
- [10] J. Manger, “A Chosen Ciphertext Attack on RSA Optimal Asymmetric Encryption Padding (OAEP) as Standardized in PKCS #1 v2.0,” em *Advances in Cryptology – CRYPTO 2001*, LNCS 2139, Springer, 2001, pp. 230–238.